UNIT-III

Sorting: Internal & external sorting, Radix sort, Quick sort, Heap sort,Merge sort,Tournament sort, Searching: Liner search, binary search, merging, Comparison of various sorting and searching algorithms on the basis of their complexity.

Difference between Internal and External sorting

Internal sorting	External sorting
1) When the entire collection of data is very small so that sorting can take place in the main memory of the computer it is called internal sorting.	1) When collection of data is large so that some of data resides on a secondary storage device such as magnetic disk or tape is called external sorting.
2) The time to read/write records is not significant in determining the performance of internal sorting.	2) Read/write access time is mainly concerned for determining the performance of sorting.
3) Internal sorting algorithms are based on the facts that memory is direct addressable.	3) External sorting algorithms are not based on the facts that memory is direct addressable.
4) Internal sorting algorithms are those algorithms that are designed to handle the input that fit into main memory i.e. which is normal sized.	4) External sorting algorithms are those algorithms that are designed to handle very large inputs that usually do not fit into main memory i.e. which is not normal sized.

RADIX SORT: -

Radix sort is one of the fastest sorting algorithms for numbers or strings of letters. This sort technique can take more space as compared to other sorting algorithms.

Since radix sort, sorts only digits or letters therefore it flexible than other sorts. Here, when names are sorted, 26 buckets are needed and when numbers are sorted, ten buckets corresponding to ten (0-9) digits are used. These buckets are represented ad first – in –first –out queues.

In case of numbers, during the first pass, the list of numbers is sorted according to the unit's digit. The numbers according to the unit digit are placed in the appropriate queues. At the end of pass, these queues are combined in proper order.

In the second pass, the numbers are sorted according to tens digit position of each number and so on. This process depends on the number of digits in the largest number. If M is the number of digits in the largest number, then M successive

passes from unit digits to the most significant digit are needed to sort the list of numbers.

ALGORITHAM: -

Let A be an array with an element

Step 1: Find the largest number of the array.

Step 2: Find the total number of digits in the largest number and denote it by K.

Step 3: Repeat step 4 and step 5 for pass=1 to K

Step 4: For i=1 to n

I. Set y□a[i]
II. Set j□ digit at the particular position of y
III. Insert y in the appropriate queue[j]

Step 5: Combine all number from the buckets in order to form a new linked list.

Step 6: Stop

Example: -

Suppose the list of number is: 131, 80, 875, 532, 12, 587, 793.

The buckets are numbered 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

In the first pass the unit's digits are stored into the buckets.

PASS 1										
	0	1	2	3	4	5	6	7	8	9
131		131								
80	80									
875						875				
532			532							
12			12							
587								587		
793				793						

The list of numbers is collected from bucket 0 to bucket 9 serially.

80, 131, 532, 12, 793, 875, 587

In the second pass the tens digits are stored into the buckets.

PASS 2										
	0	1	2	3	4	5	6	7	8	9
80									80	
131				131						
532				532						
12		12								
793										793
875								875		
587									587	

The list of numbers is collected from bucket 0 to bucket 9 serially.

12, 131, 532, 875, 80, 587, 793

In the third pass the 100th digits are stored into the buckets.

PASS 3										
	0	1	2	3	4	5	6	7	8	9
12	12									
131		131								
532					532					
875									875	
80	80									
587					587					
793								793		

After pass 3 when the numbers are collected from bucket 0 to bucket 9 serially, they are in ascending order.

12, 80, 131, 532, 587, 793, 875

In this example, there are three passes, depending upon the number of digits in largest number 875.

Merge Sort: -

- In merge sort, the concept of merging is used. The process of combining two or more sorted array into single sorted array is called merging.
- Merge sort is a sorting technique based on divide and conquer technique. With worst-case time complexity being O(n log n), it is one of the most respected algorithms.
- Merge sort first divides the array into equal halves and then combines them in a sorted manner.

- **The merge() function** is used for merging two halves. The merge(arr, l, m, r) is key process that assumes that arr[l..m] and arr[m+1..r] are sorted and merges the two sorted sub-arrays into one.

Algorithm for two way or simple merging: -

Consider two sorted array A & B having n & m elements. The sorted array contains n+m elements.

Step 1. Set i□1, j□1, k□1

Step 2. Repeat step3 while i<=n & j<=m

Step 3. If A(i)<B(j)

Then

(i) Set C(k)□ A(i)
(ii) Set i□i+1
(iii) Set k□k+1

Else if(A(i)>B(j))

(i) Set C(k)□ B(j)
(ii) Set j□j+1
(iii) Set k□k+1

Else

(i) Set C(k)□ A(i)
Set k□k+1
(ii) Set C(k)□B(j)
Set k□k+1
(iii) Set i□i+1
(iv) Set j□j+1

Step 4.

1. If i<n

Then repeat While(i<=n)

(i) Set C(k)□A(i)
(ii) Set k□k+1
(iii) Set i□i+1

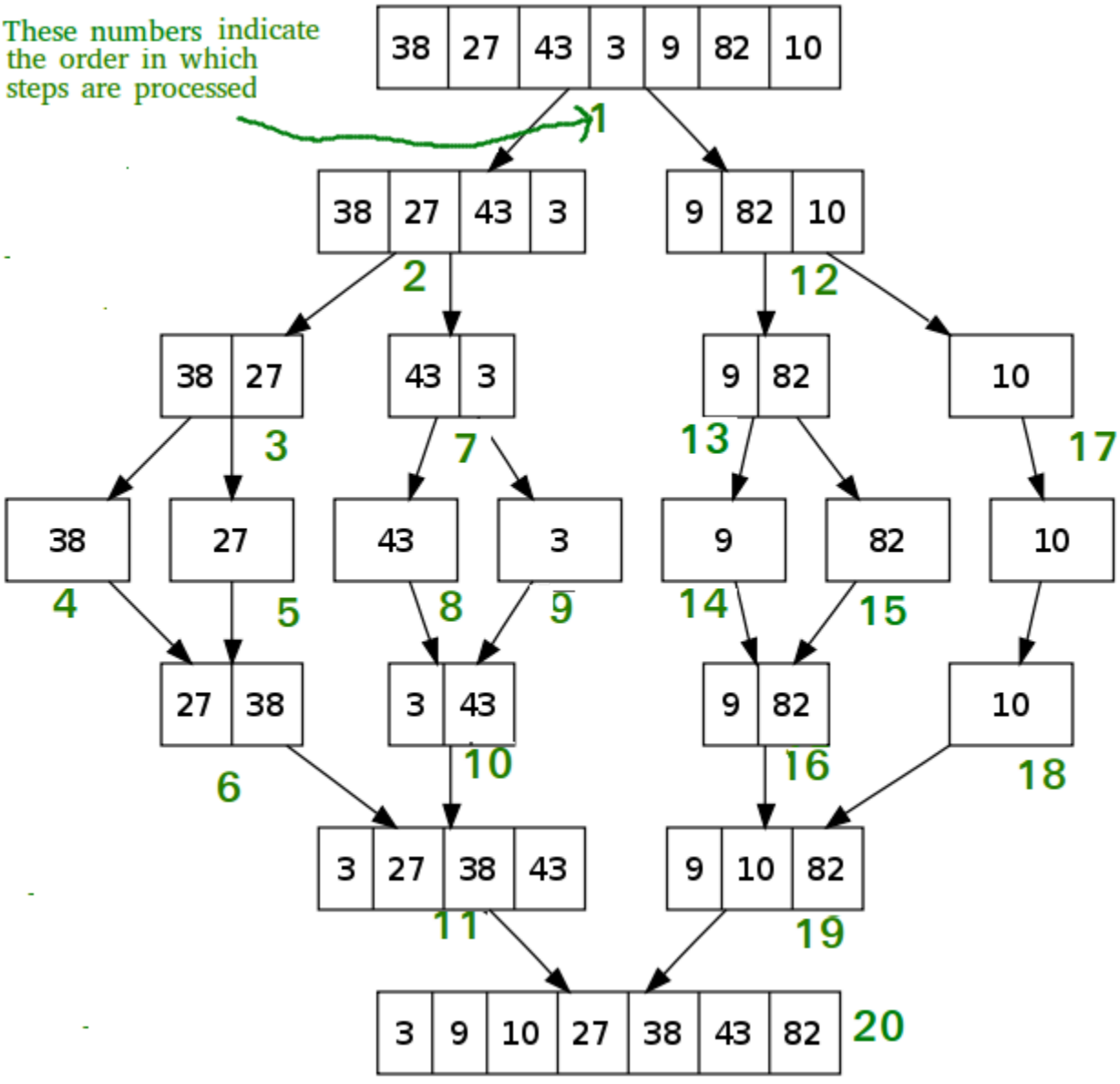
2. If j<m

Then repeat While(j<=m)

(iv) Set C(k)□B(j)
(v) Set k□k+1
(vi) Set j□j+1

Step 5. Stop

Example: -



Important Characteristics of Merge Sort:

- Merge Sort is useful for sorting linked lists.
- Merge Sort is a stable sort which means that the same element in an array maintain their original positions with respect to each other.
- Overall time complexity of Merge sort is O(nLogn). It is more efficient as it is in worst case also the runtime is O(nlogn)
- The space complexity of Merge sort is O(n). This means that this algorithm takes a lot of space and may slower down operations for the last data sets.

Quick Sort: -

- Quick sort is basically a recursive sorting techniques. In this technique first to place a particular element Ai in its final position.
- All elements having smaller values than Ai precede this element Ai, while all elements having large values follow it. This technique partition the original table into two subsets A_0, A_1, A_2,……, $A_{i-1}$and A_{i+1}, A_{i+2},…., A_{n-1}. The same process is then applied to each of these subtables and repeated until all elements are placed in their final positions.

Complexity of the Quick Sort Algorithm

To sort an unsorted list with **'n'** number of elements, we need to make **((n-1)+(n-2)+(n-3)+.....+1) = (n (n-1))/2** number of comparisons in the worst case. If the list is already sorted, then it requires **'n'** number of comparisons.

Worst Case : $O(n^2)$
Best Case : O (n log n)
Average Case : O (n log n)

Consider the following unsorted list of elements...

Define pivot, left & right. Set pivot = 0, left = 1 & right = 7. Here '7' indicates 'size-1'.

Compare List[left] with List[pivot]. If **List[left]** is greater than **List[pivot]** then stop left otherwise move left to the next.
Compare List[right] with List[pivot]. If List[right] is smaller than List[pivot] then stop right otherwise move right to the previous.
Repeat the same until **left>=right**.
If both left & right are stoped but left<right then swap List[left] with List[right] and countinue the process.
If left>=right then swap List[pivot] with List[right].

Compare List[left]<List[pivot] as it is true increment left by one and repeat the same, left will stop at 8.
Compare List[right]>List[pivot] as it is true decrement right by one and repeat the same, right will stop at 2.

Here left & right both are stoped and left is not greater than right so we need to swap List[left] and List[right]

Compare List[left]<List[pivot] as it is true increment left by one and repeat the same, left will stop at 6.
Compare List[right]>List[pivot] as it is true decrement right by one and repeat the same, right will stop at 4.

Here left & right both are stoped and left is greater than right so we need to swap List[pivot] and List[right]

Here we can observe that all the numbers to the left side of 5 are smaller and right side are greater. That means 5 is placed in its correct position.
Repeat the same process on the left sublist and right sublist to the number 5.

In the left sublist as there are no smaller number than the pivot left will keep on moving to the next and stops at last number. As the List[right] is smaller, right stops at same position. Now left and right both are equal so we swap pivot with right.

In the right sublist left is grester than the pivot, left will stop at same position.
As the List[right] is greater than List[pivot], right moves towords left and stops at pivot number position.
Now left > right so we swap pivot with right. (6 is swap by itself).

Repeat the same recursively on both left and right sublists until all the numbers are sorted.
The final sorted list will be as follows...

Example: -consider the placement of A[0] element to its final position in the array of 10 elements given below:

64, 20, 75, 9, 99, 31, 97, 42, 86, 53

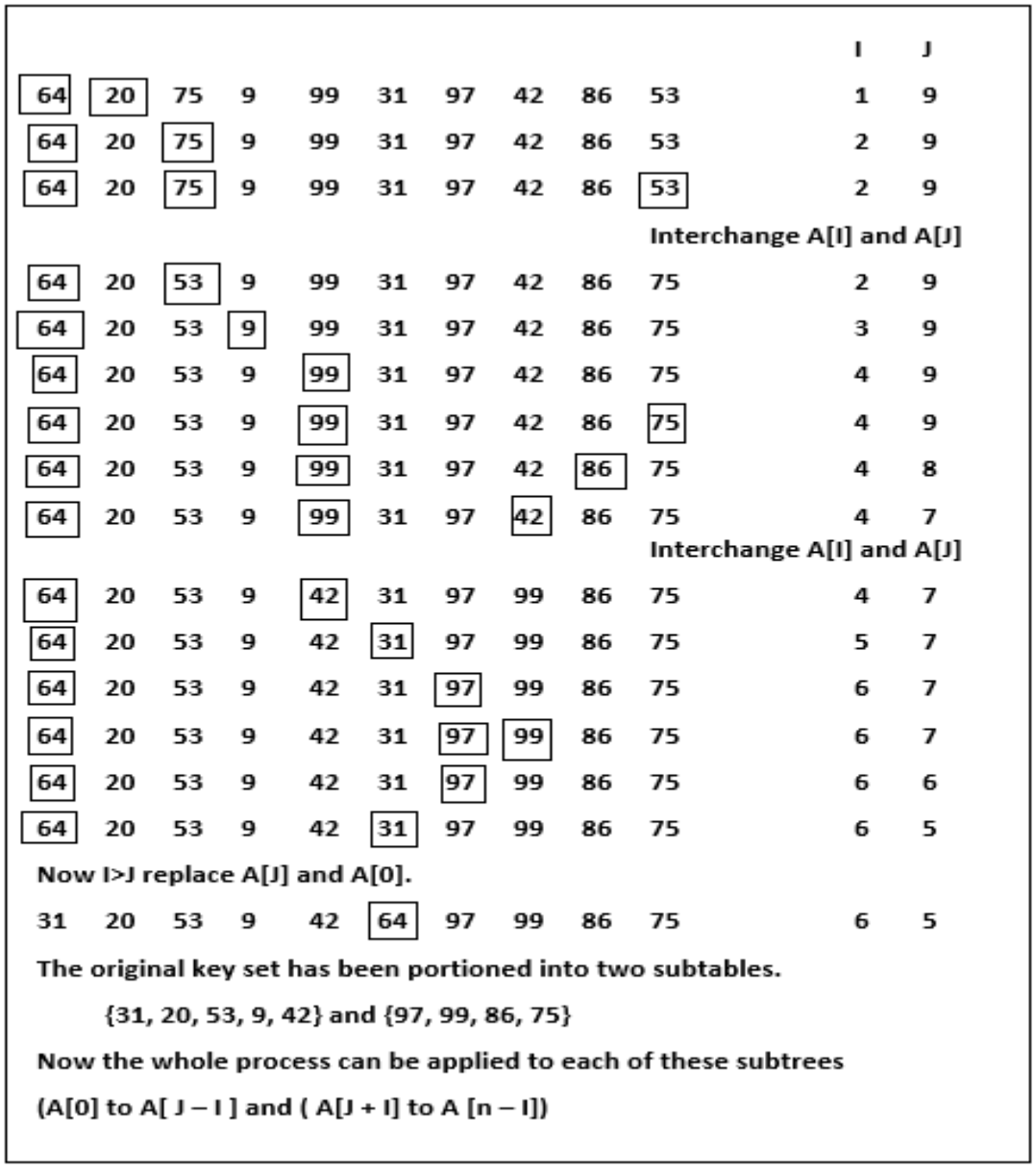
Two index variables I and J with initial values of 1 and 9 respectively are used. A[0] is compared with A[I].

- If A[I]<A[0] then I is incremented by 1 and the process is repeated.
- If A[I]>=A[0] then A[J] and A[0] are compared.
- If A[J]>A[0] then J is decremented by 1 and the process is repeated. If A[J]<A[0] then A[I] and A[J] are interchanged.

The entire process is then repeated. When I>J, the desired key is placed in its final position by interchanging A[0] and A[J].

Define pivot, left and right. Set pivot= 0, left= 1, right=9. Here "**9**" indicates "**size-1**".

Algorithm: -



Tournament Tree

Tournament tree is a complete binary tree n external nodes and n-1 internal nodes. The external nodes represent the players and internal nodes represent the winner of the match between the two players.

- The tournament tree is described by a binary tree
  - Each external node represents a player
  - Each internal node represents a match played
  - Each level of internal nodes defines a round of matches
- Tournament trees are also called selection trees

Properties of Tournament Tree

1. It is rooted tree i.e. the links in the tree and directed from parents to children and there is a unique element with no parents.
2. The key value of the parent node has less than or equal to that node to general any comparison operators can be used as long as the relative values of parent-child are invariant throughout the tree. The tree is a parent ordering of the keys.
3. Trees with a number of nodes not a power of 2 contain holes which is general may be anywhere in the tree.
4. Tournament tree is a proper generalization of heaps which restrict a node to at most two children.
5. The tournament tree is also called selection tree.
6. The root of the tournament tree represents overall winner of the tournament

Types of tournament Tree

There are mainly two type of tournament tree,

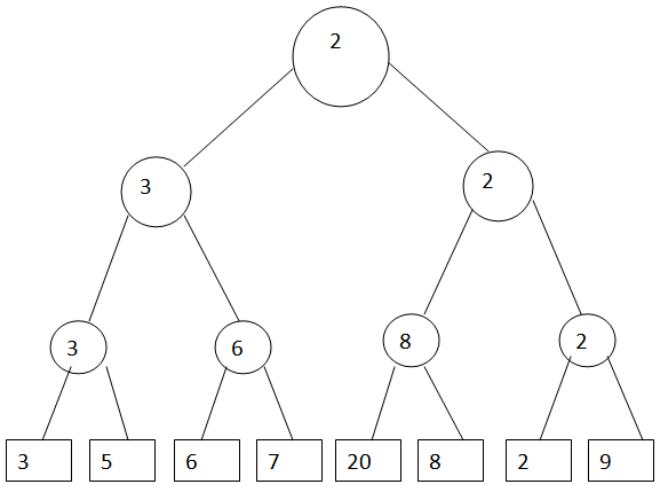
1. **Winner tree**
2. **Loser tree**

1) Winner tree

- The complete binary tree in which each node represents the smaller or greater of its two children is called a winner tree.
- The smallest or greater node in the tree is represented by the root of the tree.
- The winner of the tournament tree is the smallest or greatest n key in all the sequences.
- It is easy to see that the winner tree can be computed in **O(logn)** time.

Example: Consider some keys 3, 5, 6, 7, 20, 8, 2, 9

We try to make minimum or maximum winner tree



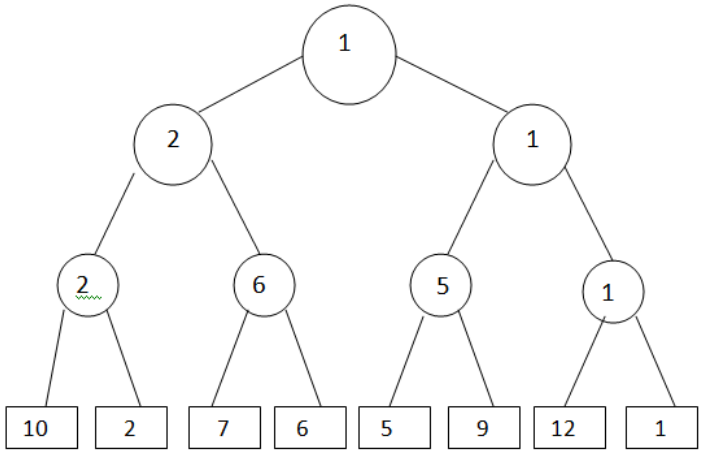
Winner Tree(Minimum)

2) Loser Tree

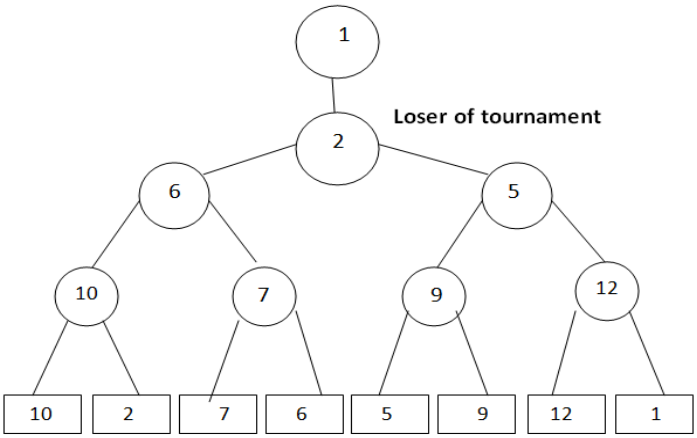
- The complete binary tree for n players in which there are n external nodes and n-1 internal nodes then the tree is called loser tree.
- The loser of the match is stored in internal nodes of the tree. But in this overall winner of the tournament is stored at tree [0].
- The loser is an alternative representation that stores the loser of a match at the corresponding node.
- An advantage of the loser is that to restructure the tree after a winner tree been output, it is sufficient to examine node on the path from the leaf to the root rather than the sibling of nodes on this path.

Example: Consider some keys 10, 2, 7, 6, 5, 9, 12, 1

Step 1) We will first draw min winner tree for given data.



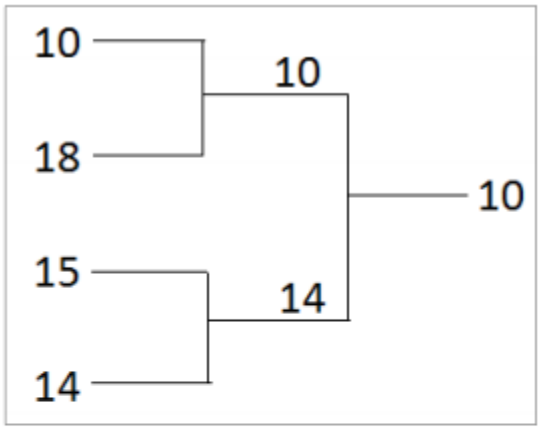
Step 2) Now we will store losers of the match in each internal nodes.



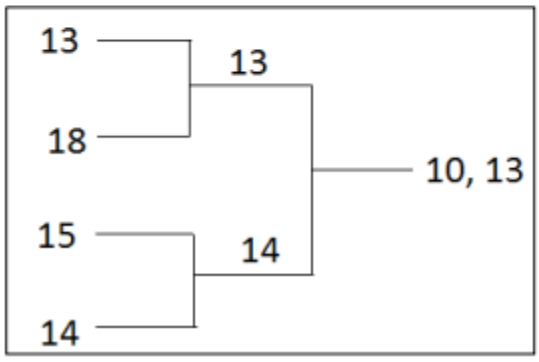
Loser Tree

Example: Unsorted List 10, 18, 15, 14, 13, 21, 28, 11, 12, 30

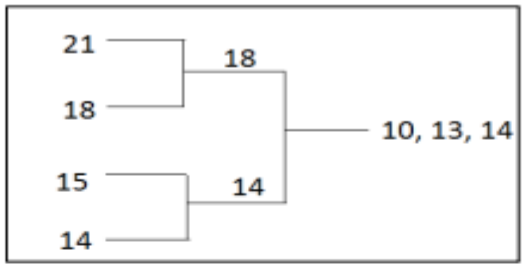
Pass 1:



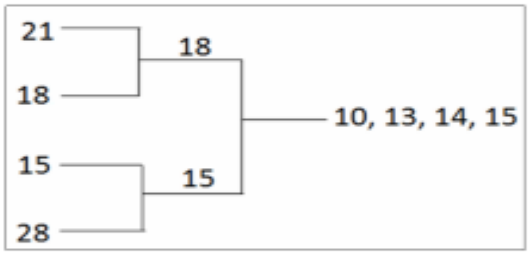
Pass 2:



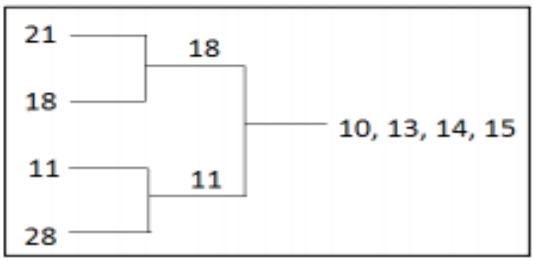
Pass 3:



Pass 4:

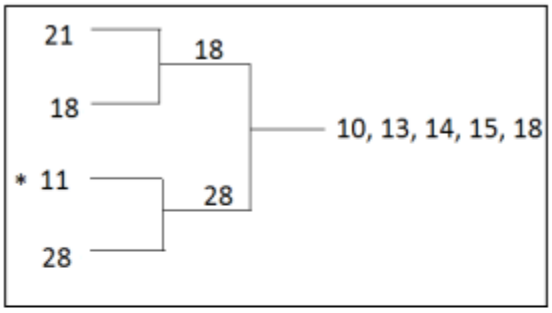


Pass 5:

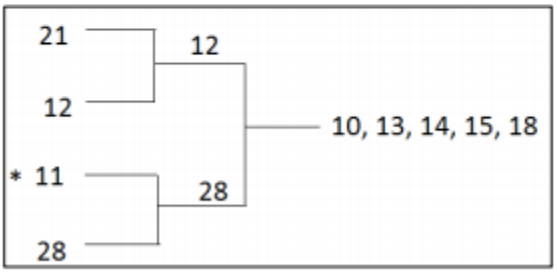


Now if we put 11 in the sequence, the sorted sequence will be broken. Hence we have to disqualify the element by putting * mark on it. Element is disqualified if key (new)<key(last out)

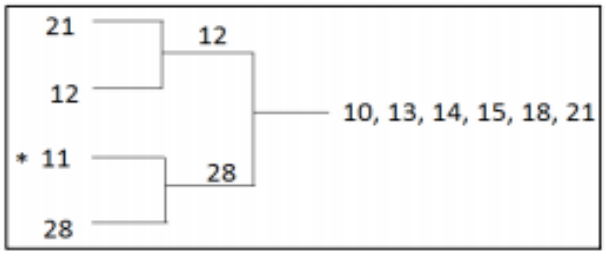
So Pass 5:



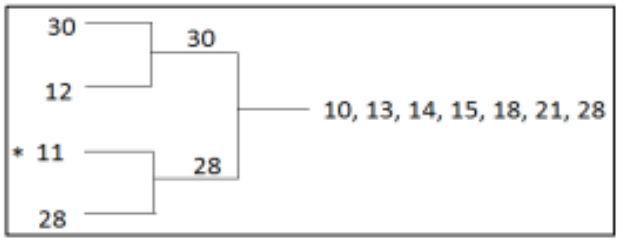
Pass 6:



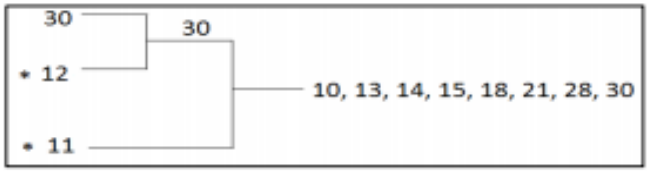
12 is disqualified
So



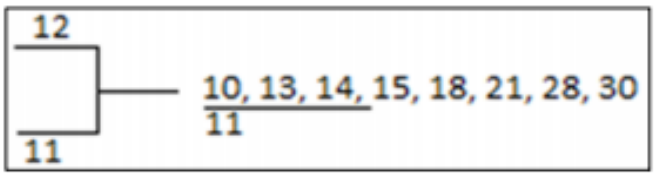
Pass 7:



Pass 8:



Pass 9: Make a separate list of the disqualified elements, we get



Pass 10: We have 2 sorted lists:

I. 10, 13, 14, 15, 18, 21, 28, 30
II. 11, 12

Now, merge both the sub lists to get a sorted list

10,11,12,13,14,15,18,21,28,30

Complexity:

Worst case – O (n log n)

Average case- O (n log n)

Heap sort: -

Heap Sort is a popular and efficient sorting algorithm in computer programming. Learning how to write the heap sort algorithm requires knowledge of two types of data structures - arrays and trees.

Heap property is a binary tree with special characteristics. It can be classified into two types:

I. Max-Heap
II. Min Heap

I. Max Heap: If the parent nodes are greater than their child nodes, it is called a Max-Heap.

II. Min Heap: If the parent nodes are smaller than their child nodes, it is called a Min-Heap.

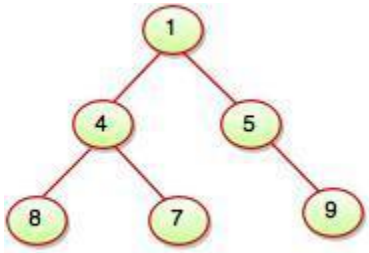


Fig. Min Heap

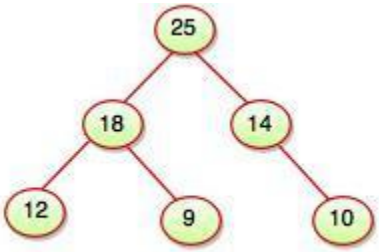


Fig. Max Heap

What is heap?

A heap of size n is a binary tree of n nodes with the following conditions:

a) The binary tree is almost complete which means:
   - All the children of the tree are on two adjacent levels i.e. there exist an integer k such that all children of the tree are at level k or k+1.
   - The level of the heap are filled from left to right and a node is not placed on a new level if the preceding level is not full.

b) The value of the node at the root of any subtree is large than or equal to the value of either of its children. This means that for every node info[J]>= info[I] where J is the father of I.

The heap sort consists of two phase:

1. **Creation of heap**
2. **Processing of heap**

1. Creation of heap:-

- Place the child at the leaf level.
- Obtain position of parent for this child.

- Repeat step (a) and (b) while the child has a parent and the data value of the child is greater than that of the parent:

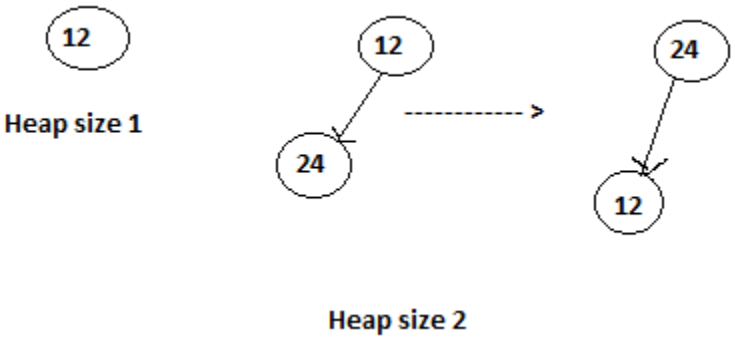
(a) Move the parent down to the position of the child.

(b) Obtain the position of the new parent for the child.

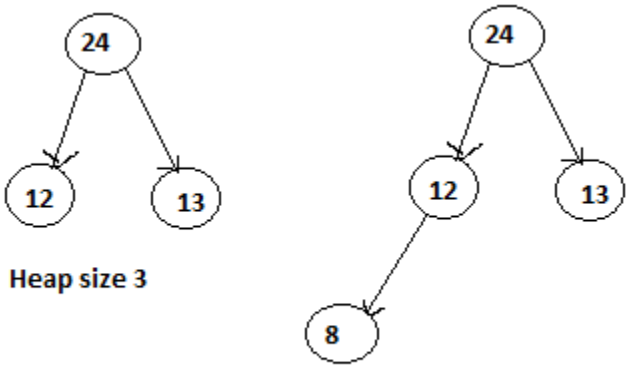
- Copy the data value of child in its proper place.

Example: - Consider the following unsorted list of records:

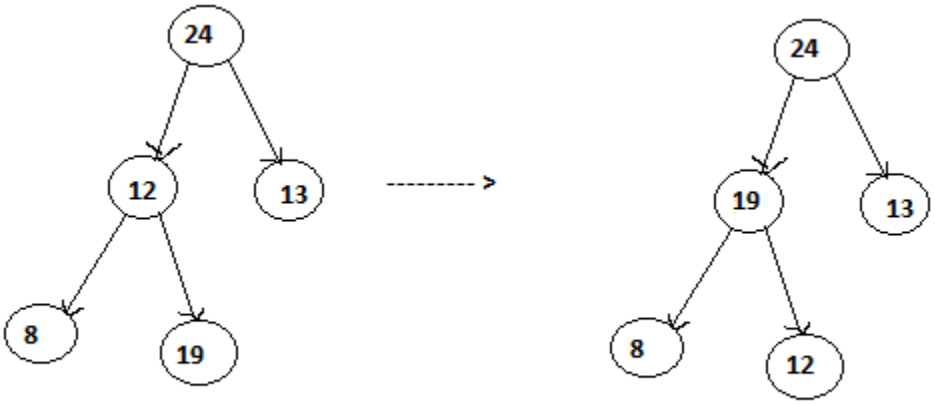
12 24 13 8 19 14 11 26 7 16



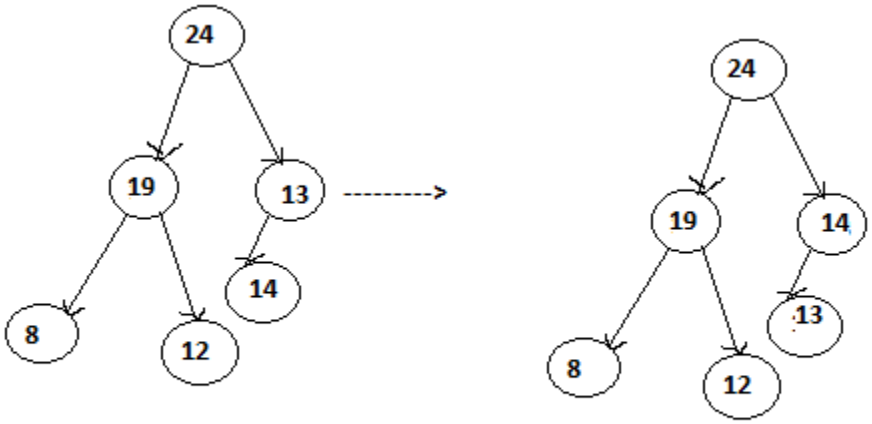
Heap size 2



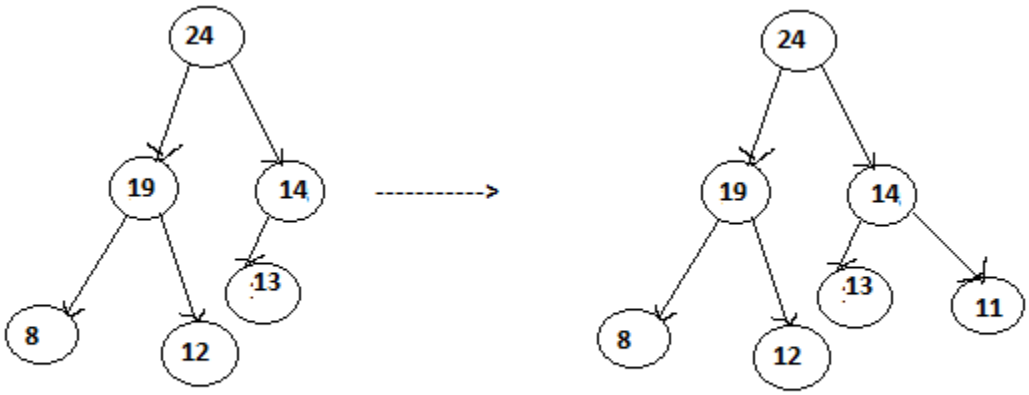
Heap size 4



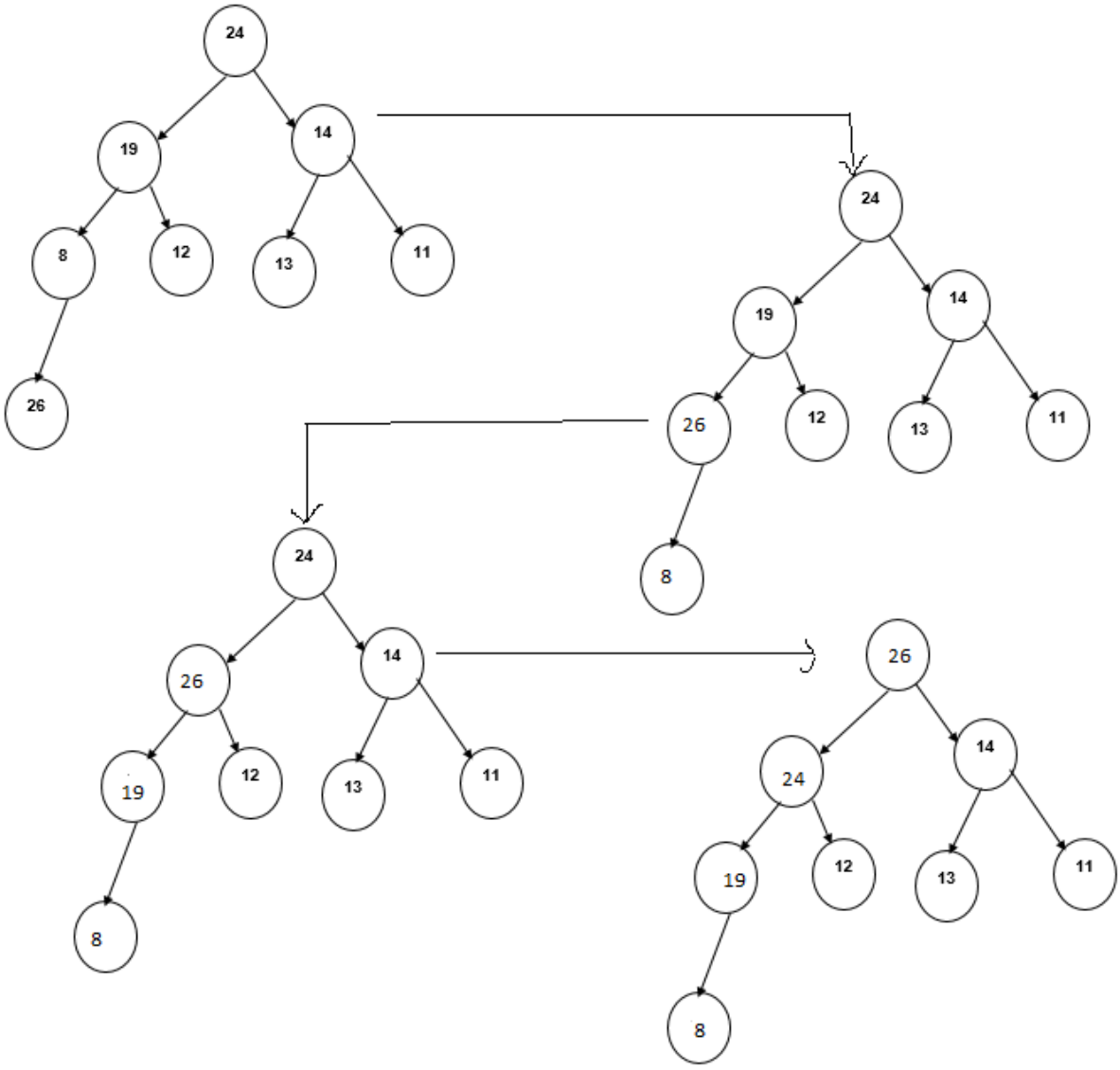
Heap size 5



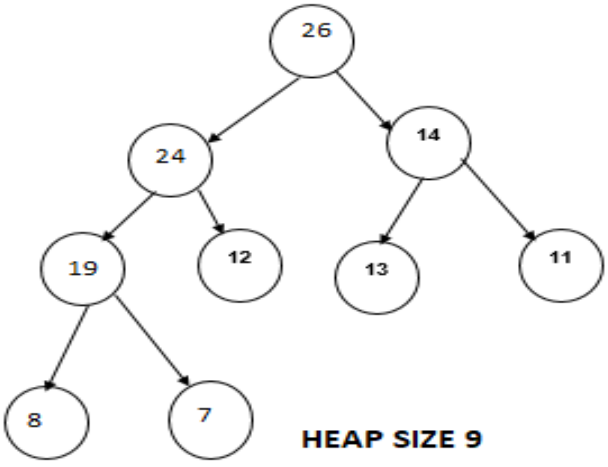
Heap size 6



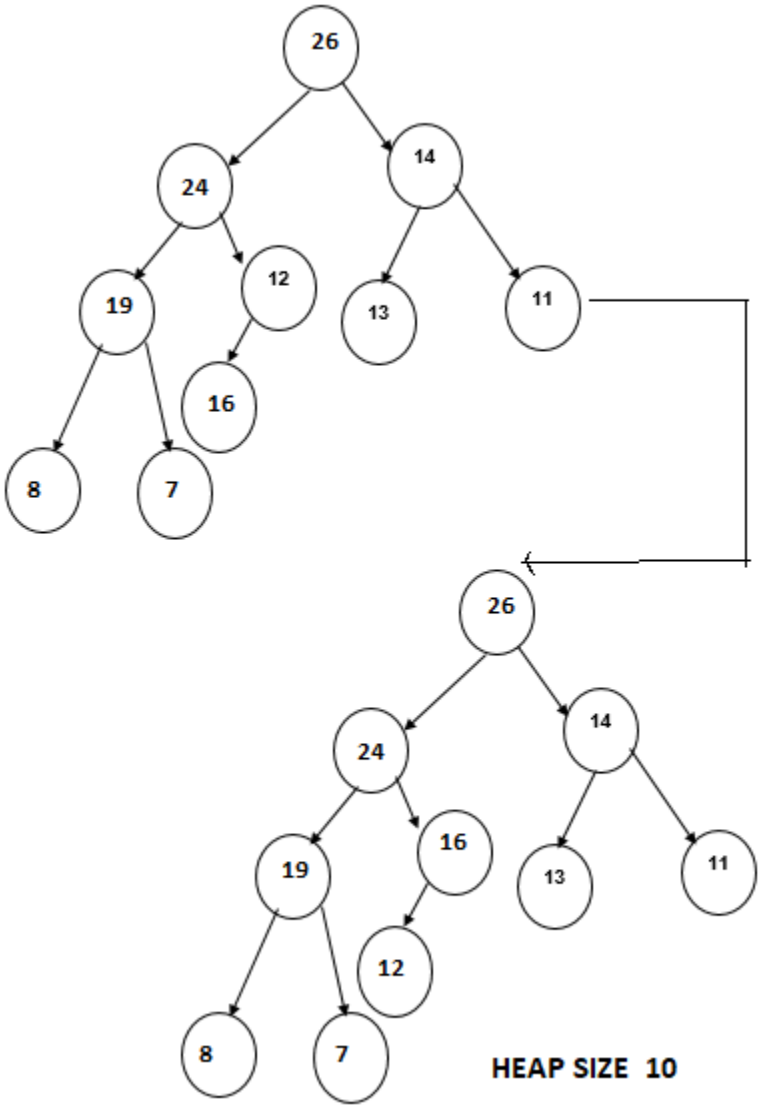
Heap size 7



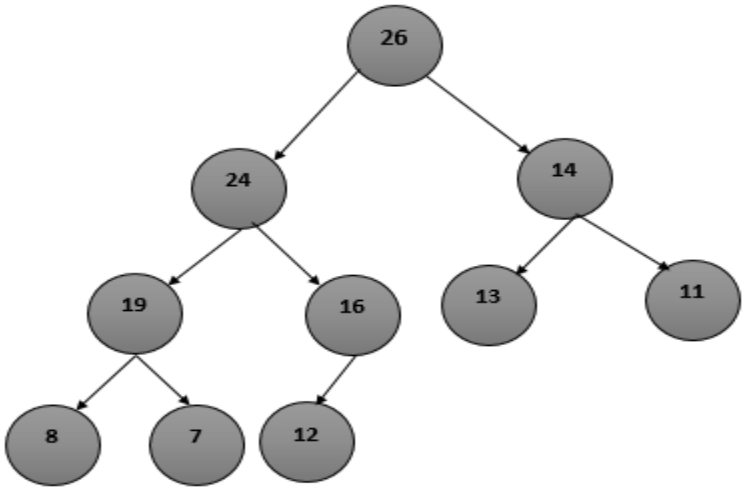
HEAP SIZE 8



HEAP SIZE 9

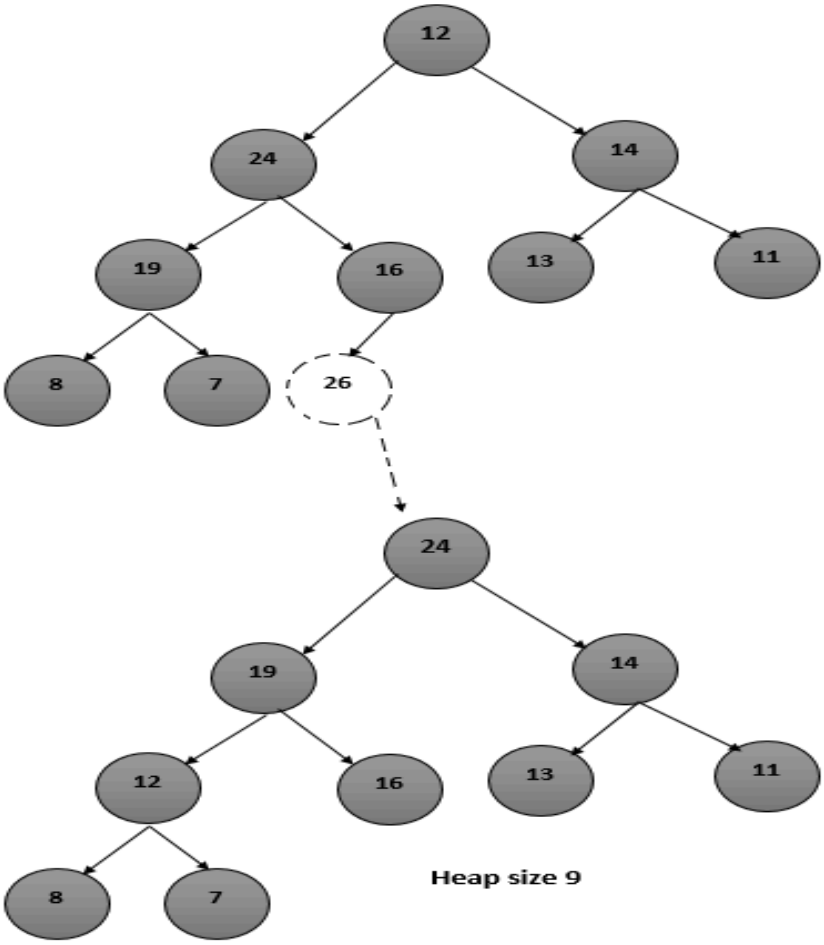


2. **Processing a heap:-**
   Traverse the heap in such a way so that the sorted keys are outputted. The largest value is at the top of the heap which is sorted in the array at position a[1]. Interchange it with the last position i.e. a[n]. Then adjust the array to be a heap of size n-1.
   Again interchange a[1] with a[n-1], adjusting the array to be a heap of size and so on. At the end, we get an array which contains the value in the sorted order.
   EXAMPLE: -
   Consider the created heap of size 10



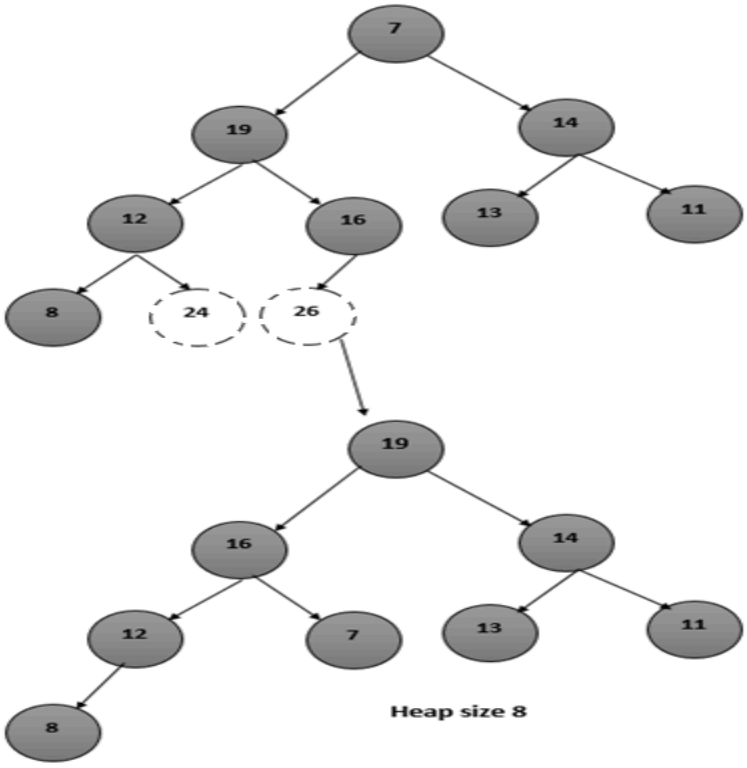
Heap size 10

Interchange a[1] with the last position i.e. a[10]. Then adjust the array to be a heap of size 9.

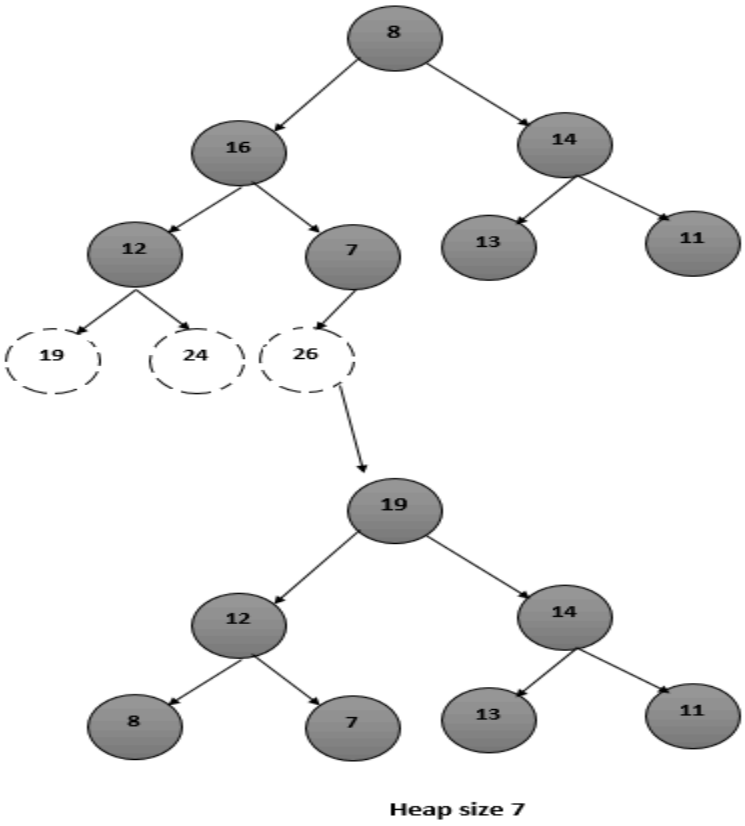


Heap size 9

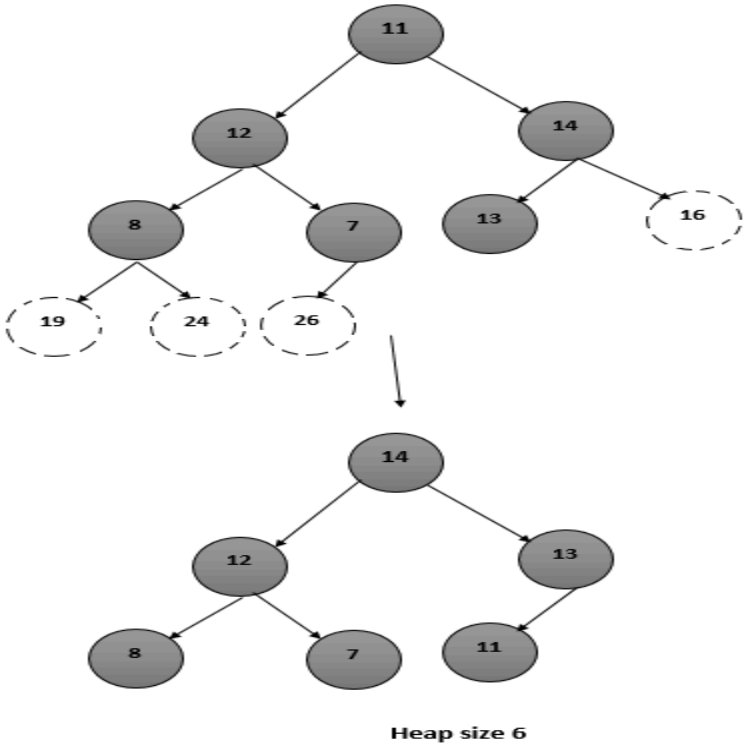
Now Interchange a[1] with a[9]. Then adjust the array to be a heap of size 8.



Now Interchange a[1] with a[8]. Then adjust the array to be a heap of size 7.

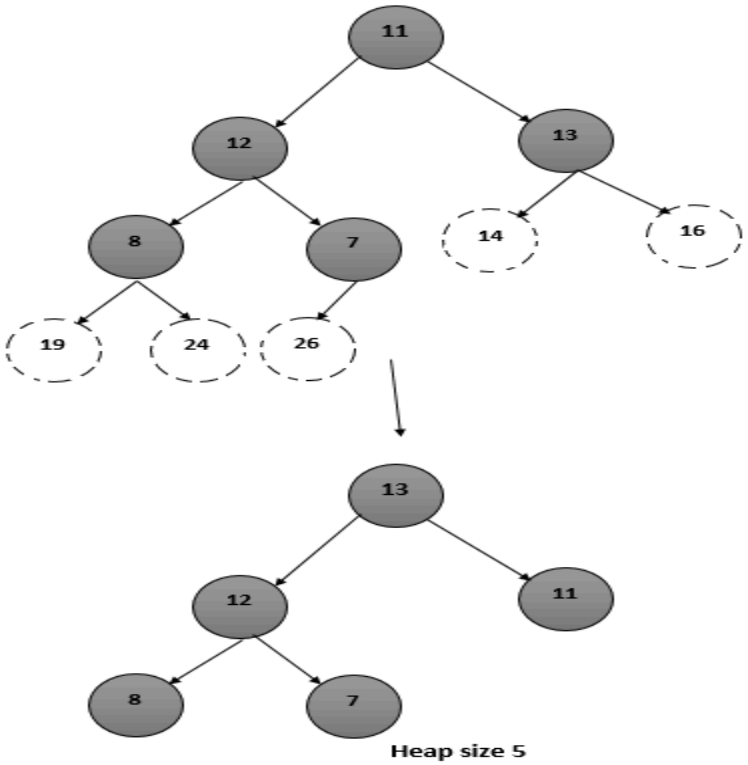


Now Interchange a[1] with a[7]. Then adjust the array to be a heap of size 6.

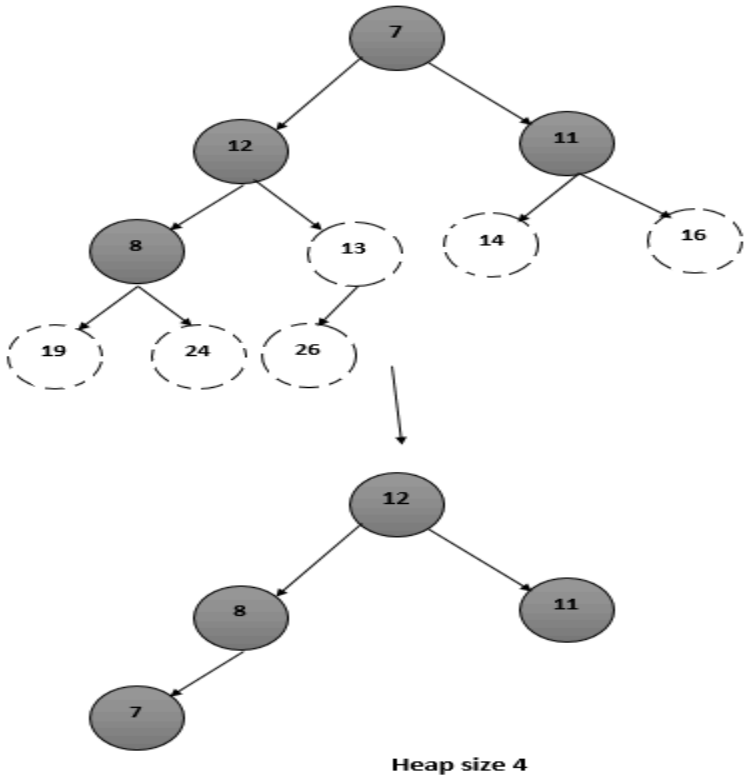


Heap size 6

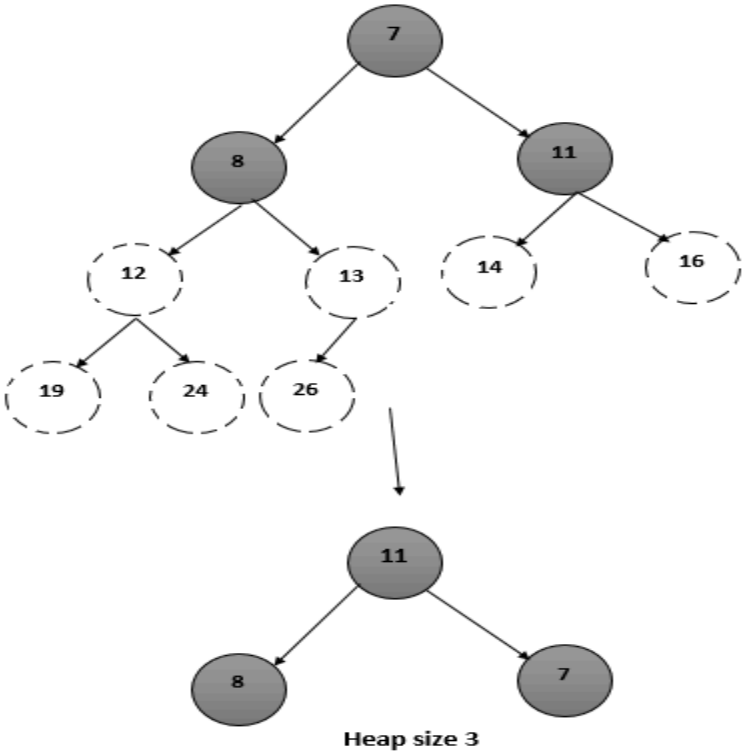
Now Interchange a[1] with a[6]. Then adjust the array to be a heap of size 5.



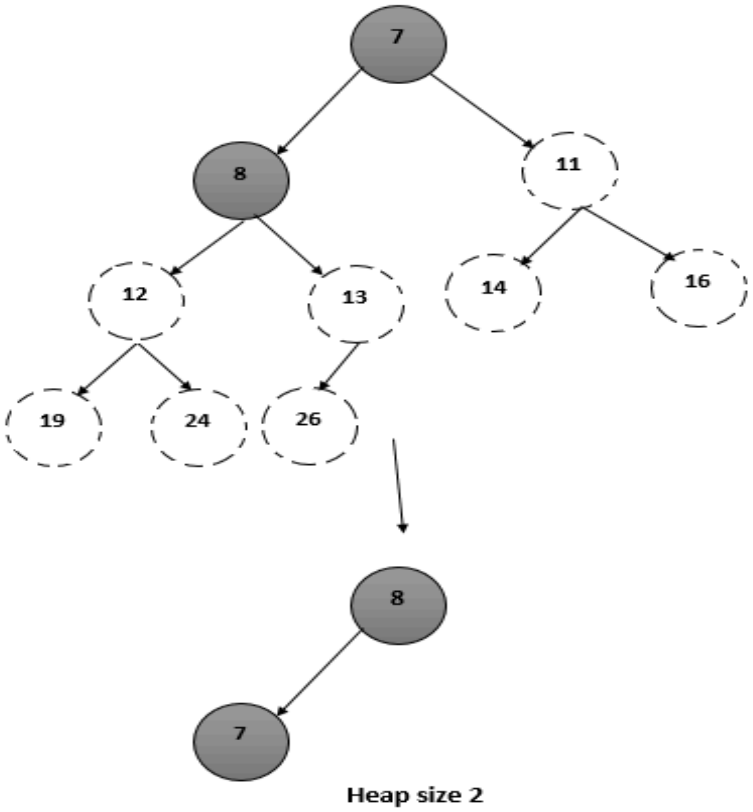
Now Interchange a[1] with a[5]. Then adjust the array to be a heap of size 4.



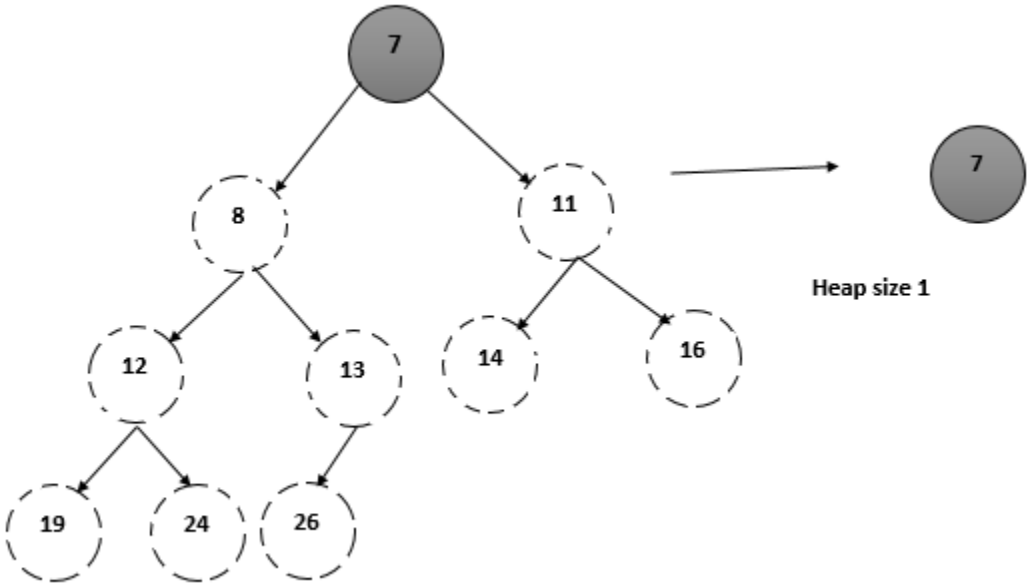
Now Interchange a[1] with a[4]. Then adjust the array to be a heap of size 3.



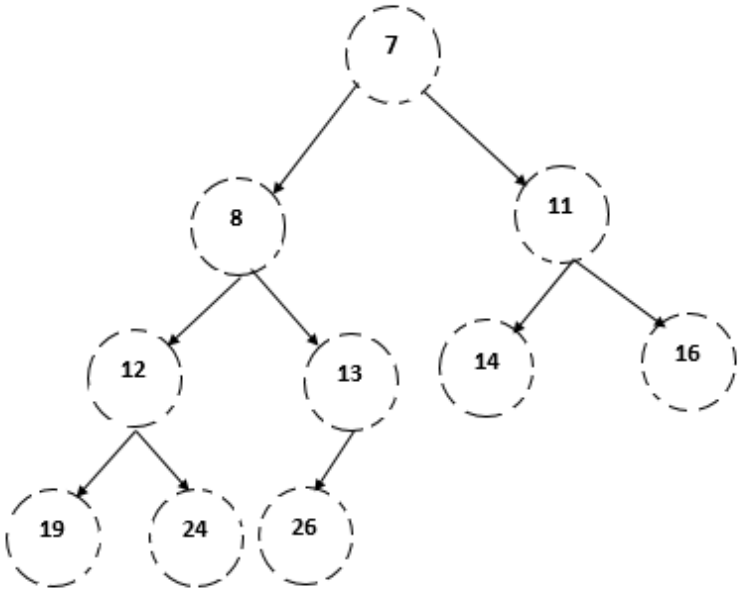
Now Interchange a[1] with a[3]. Then adjust the array to be a heap of size 2.



Now Interchange a[1] with a[2]. Then adjust the array to be a heap of size 1.



Now all the keys are sorted.



Searching: -

- Searching is a process of finding a particular element among several given elements.
- The search is said to be successful if the required element is found otherwise the search is unsuccessful.

- Searching Algorithms are a family of algorithms used for the purpose of searching.

Approaches to searching-

The searching of an element in a given array can be done in two ways-

2. Linear Search
3. Binary Search

What is Linear Search?

- A Linear Search is the most basic type of searching algorithm. A Linear Search sequentially moves through your collection (or data structure) looking for a matching value.
- Linear Search is the simplest approach to searching which searches for an element by comparing it with each element of the array one by one.
- When the required element is successfully found, it returns the index of that element in the array, else it returns -1.
- Because linear search traverses the array sequentially to locate the element, it is also known as **Sequential Search**.

Linear search is implemented using following steps...

- **Step 1 -** Read the search element from the user.
- **Step 2 -** Compare the search element with the first element in the list.
- **Step 3 -** If both are matched, then display "Given element is found!!!" and terminate the function
- **Step 4 -** If both are not matched, then compare search element with the next element in the list.
- **Step 5 -** Repeat steps 3 and 4 until search element is compared with last element in the list.
- **Step 6 -** If last element in the list also doesn't match, then display "Element is not found!!!" and terminate the function.

Algorithm for linear search:-

Let A be a linear array of n elements. The algorithm searches for the element DATA. LOC represents the location of the element DATA in the array.

The algorithm returns the value LOC=0 if the element DATA does not belong to the array.

ALGORITHAM:-

Step 1: Set K□1.

Step 2: Set LOC□0.

Step 3: Repeat steps 4 and 5 while LOC=0 and K<=n

Step 4: If A(K)= DATA then Set LOC□K.

Step 5: Set K□ K+1

Step 6: If LOC=0 then write DATA not found in A

Else write DATA found at LOC in A

Step 7: End.

Advantages of Linear Search

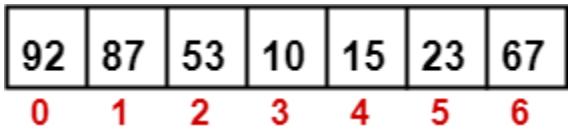
- It is very simple method.
- It is easy to understand and easy to implement.
- This is the best method if numbers of elements are less.
- The numbers need not be in any particular order.

Disadvantages of linear search:

- It is not suitable if the numbers of elements are more.
- If the element is at last position then maximum comparisons are required.

1. Example of Linear Search-

Consider we are given the following linear array and we have to search for element 15 in it.



We will compare the element to be searched (element 15) with all the elements of the given linear array one by one until we find the element 15 or all the elements are searched.

Step-01:

We compare element 15 with the 1^{st} element 92. Since, $15 \neq 92$, so required element is not found. So, we move to the next element.

Step-02:

We compare element 15 with the 2^{nd} element 87. Since, $15 \neq 87$, so required element is not found. So, we move to the next element.

Step-03:

We compare element 15 with the 3rd element 53. Since, 15 ≠ 53, so required element is not found. So, we move to the next element.

Step-04:

We compare element 15 with the 4th element 10. Since, 15 ≠ 10, so required element is not found. So, we move to the next element.

Step-05:

We compare element 15 with the 5th element 15. Since, 15 = 15, so required element is found.

Now, we stop the comparison and return index 4 at which the element being searched is present.

2. Example

Consider the following list of elements and the element to be searched...

Step 1:

search element (12) is compared with first element (65)

Both are not matching. So move to next element

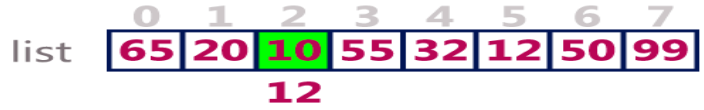
Step 2:

search element (12) is compared with next element (20)

Both are not matching. So move to next element

Step 3:

search element (12) is compared with next element (10)



Both are not matching. So move to next element

Step 4:

search element (12) is compared with next element (55)

Both are not matching. So move to next element

Step 5:

search element (12) is compared with next element (32)

Both are not matching. So move to next element

Step 6:

search element (12) is compared with next element (12)

Both are matching. So we stop comparing and display element found at index 5.

Binary Search:-

- **Binary search** is the most popular Search algorithm. It is efficient and also one of the most commonly used techniques that are used to solve problems.
- Binary search works only on a sorted set of elements. To use binary search on a collection, the collection must first be sorted.

The procedure for the binary search is as follows:

- First to find the middle element of the array. For this divide the whole range of elements from LB(LOW) to UB(HIGH) into two halves. Let the middle be A(MID) where MID= (LOW+HIGH)/2.
- After locating the middle element,A(MID), it is compared with DATA i.e. the required element to be searched.
- If A(MID)= DATA, the search is successful, otherwise a new range, left half or right half is considered and searched again.
- If DATA<A(MID) then DATA lies in first range. Reset HIGH=MID-1 and begin the search again.
- If DATA>A(MID) then DATA lies in second range. Reset HIGH=MID+1 and begin the search again.

Algorithm:-

Step 1: Set LOW□LB

Step 2: Set HIGH□UB

Step 3: FOUND□N.

Step 4: While FOUND=N and LOW<= HIGH execute step a, b, c & d below

a) MID= (LOW+HIGH)/2
b) If DATA>A(MID) then Set LOW□ MID+1
c) If DATA<A(MID) then Set LOW□ MID-1
d) If DATA=A(MID) then Set FOUND□Y

Step 5: If FOUND=Y, write DATA found in location MID

Step 6: If FOUND=N, write DATA not found.

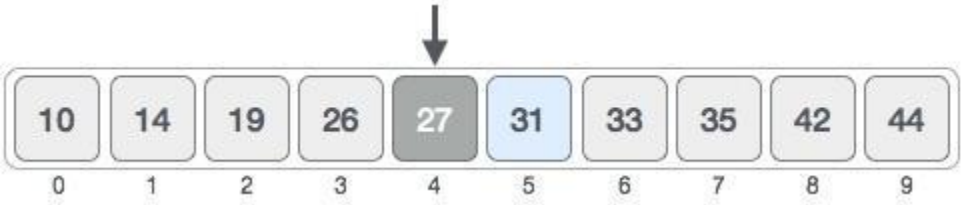
Step 7: Stop.

Example:-

First, we shall determine half of the array by using this formula –

Mid=low + (high-low)/2

Here it is, 0 + (9 - 0) / 2 = 4 (integer value of 4.5). So, 4 is the mid of the array.



Now we compare the value stored at location 4, with the value being searched, i.e. 31. We find that the value at location 4 is 27, which is not a match. As the value is greater than 27 and we have a sorted array, so we also know that the target value must be in the upper portion of the array.

We change our low to mid + 1 and find the new mid value again.

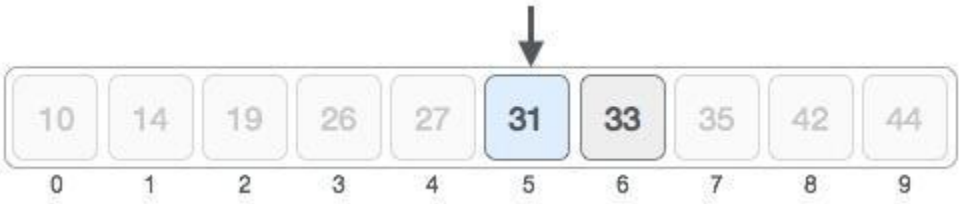
Low=mid+1

Mid=low+(high-low)/2

Our new mid is 7 now. We compare the value stored at location 7 with our target value 31.

The value stored at location 7 is not a match, rather it is more than what we are looking for. So, the value must be in the lower part from this location.

Hence, we calculate the mid again. This time it is 5.



We compare the value stored at location 5 with our target value. We find that it is a match.

We conclude that the target value 31 is stored at location 5.

Example:-

	0	1	2	3	4	5	6	7	8
List	10	12	20	32	50	55	65	80	99

SEARCH ELEMENT 12

STEP 1:

Search element (12) is compared with middle element (50)

	0	1	2	3	4	5	6	7	8
List	10	12	20	32	50	55	65	80	99
					12				

Both are not matching, and 12 is smaller than 50. So we search only in the left sublist (i.e. 10, 12, 20 & 32).

	0	1	2	3	4	5	6	7	8
List	10	12	20	32	50	55	65	80	99

STEP 2:

Search element (12) is compared with middle element (12)

	0	1	2	3	4	5	6	7	8
List	10	12	20	32	50	55	65	80	99
		12							

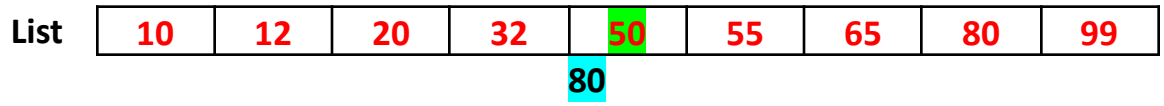
Both are matching. So the result is “element found at index 1”

SEARCH ELEMENT 80

STEP 1:

Search element (80) is compared with middle element (50)

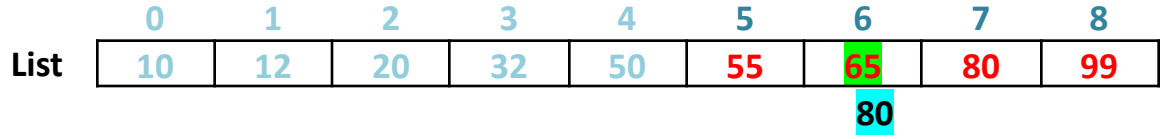
0	1	2	3	4	5	6	7	8



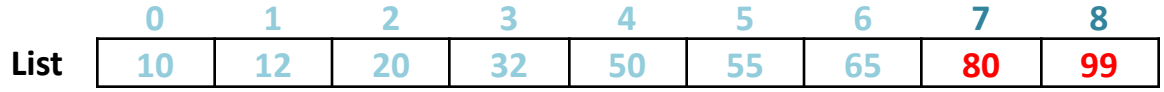
Both are not matching, and 80 is larger than 50. So we search only in the right sublist (i.e. 55, 65, 80 & 99).

STEP 2:

Search element (80) is compared with middle element (65)

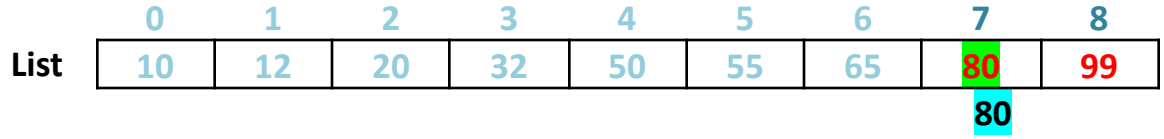


Both are not matching, and 80 is larger than 65. So we search only in the right sublist (i.e. 80 & 99).



STEP 3:

Search element (80) is compared with middle element (80)



Both are matching. So the result is "element found at index 7"

Comparison of various sorting and searching algorithms on the basis of their complexity

Algorithm	Best Time Complexity	Average Time Complexity	Worst Time Complexity	Worst Space Complexity
Linear Search	O(1)	O(n)	O(n)	O(1)
Binary Search	O(1)	O(log n)	O(log n)	O(1)
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Merge Sort	O(nlogn)	O(nlogn)	O(nlogn)	O(n)
Quick Sort	O(nlogn)	O(nlogn)	O(n^2)	O(log n)
Heap Sort	O(nlogn)	O(nlogn)	O(nlogn)	O(n)
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)